JUNIT

CHEAT SHEETS

Testing Java with JUnit 6, AssertJ and Mockito

Platform & Jupiter · Maven & Gradle · Lifecycle · @Nested · @Tag · Assertions · AssertJ · Assumptions · @ParameterizedTest · @MethodSource · @TestFactory · Extensions · @TempDir · Parallel · Mockito · @SpringBootTest · Test Slices · Testcontainers · Migration · CI Reports

25 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & Architecture
02 Setup with Maven & Gradle
03 Running Tests

WRITING TESTS

04 Test Classes & Methods
05 Lifecycle
06 Nested, Tagged & Ordered Tests

ASSERTIONS & CONDITIONS

07 Core Assertions
08 Exceptions, Grouped Assertions & Timeouts
09 AssertJ
10 Assumptions & Conditional Tests

PARAMETERIZED & DYNAMIC TESTS

11 Parameterized Tests — Sources
12 Method Sources & Argument Providers
13 Dynamic & Repeated Tests

EXTENSIONS & EXECUTION

14 The Extension Model
15 Built-in Parameters & Extensions
16 Parallel Execution

MOCKING WITH MOCKITO

17 Mockito Basics
18 Mockito Advanced

SPRING BOOT & INTEGRATION TESTS

19 Spring Boot Test Basics
20 Spring Test Slices
21 Testcontainers

MIGRATION & CI

22 Migrating from JUnit 4 & 5
23 Reports, Coverage & CI

QUICK REFERENCE

24 Quick Reference — Annotations & Assertions
25 Quick Reference — Mockito, Build & Config

JU

Table of Contents

25 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 03

- 01 Introduction & Archite…
- 02 Setup with Maven & Gr…
- 03 Running Tests

WRITING TESTS 04 – 06

- 04 Test Classes & Methods
- 05 Lifecycle
- 06 Nested, Tagged & Ord…

ASSERTIONS & CONDITIONS 07 – 10

- 07 Core Assertions
- 08 Exceptions, Grouped A…
- 09 AssertJ
- 10 Assumptions & Condit…

PARAMETERIZED & DYNAMIC TESTS 11 – 13

- 11 Parameterized Tests —…
- 12 Method Sources & Arg…
- 13 Dynamic & Repeated T…

EXTENSIONS & EXECUTION 14 – 16

- 14 The Extension Model
- 15 Built-in Parameters & …
- 16 Parallel Execution

MOCKING WITH MOCKITO 17 – 18

- 17 Mockito Basics
- 18 Mockito Advanced

SPRING BOOT & INTEGRATION TESTS 19 – 21

- 19 Spring Boot Test Basics
- 20 Spring Test Slices
- 21 Testcontainers

MIGRATION & CI 22 – 23

- 22 Migrating from JUnit 4 …
- 23 Reports, Coverage & CI

QUICK REFERENCE 24 – 25

- 24 Quick Reference — An…
- 25 Quick Reference — Mo…

Sheet 01

Introduction & Architecture

JUnit Platform · Jupiter · Vintage · test engines · JUnit 5 vs 6 · Java 17 baseline · first test class

A PLATFORM, AN API AND ENGINES

JUnit is the standard test framework for Java. Since JUnit 5 it is split in three: the **Platform** launches tests and talks to IDEs and build tools, **Jupiter** is the API you write tests with (`@Test`, `Assertions`), and **Vintage** runs old JUnit 4 tests. Other **engines** (Spock, Cucumber, Kotest) plug into the same Platform. **JUnit 6** keeps the Jupiter API, raises the baseline to Java 17 and gives every module one version number.

NOTES

- Imports: `org.junit.jupiter.api.*`
- Test classes and methods can be package-private
- JUnit 6 deprecates the Vintage engine; migrate JUnit 4 tests (sheet 22)
- Most JUnit 5 tests run unchanged on JUnit 6

FIRST TEST CLASS

```java
// src/main/java/shop/Cart.java
public class Cart {
    private final Map<String, Integer> items = new HashMap<>();
    public void add(String sku, int qty) {
        if (qty <= 0) throw new IllegalArgumentException("qty must be > 0");
        items.merge(sku, qty, Integer::sum);
    }
    public int count(String sku) { return items.getOrDefault(sku, 0); }
}

// src/test/java/shop/CartTest.java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CartTest {
    @Test
    void addsNewItem() {
        Cart cart = new Cart();
        cart.add("A1", 2);
        assertEquals(2, cart.count("A1"));
    }

    @Test
    void rejectsZeroQuantity() {
        assertThrows(IllegalArgumentException.class, () -> new Cart().add("A1", 0));
    }
}
```

CODE EXPLAINED

- → Tests live in `src/test/java` in the same package as the class they test
- → `@Test` marks a test method; it returns `void` and takes no arguments (unless resolved, sheet 14)
- → `assertEquals(expected, actual)` puts the expected value first
- → A new instance of `CartTest` is created for each test method

JUNIT 4 / TESTNG EQUIVALENT

- ↺ JUnit 4 `org.junit.Test` on `public` methods in a `public` class; `org.junit.Assert.*`
- ↺ TestNG `org.testng.annotations.Test`; `Assert.assertEquals(actual, expected)` (actual first!)

PREFER / AVOID

- ⇄ `public class` / `public void` tests → package-private (less noise)
- ⇄ `System.out.println` checks → assertions
- ⇄ one test per class method → one test per behavior

MODULES

PART	ARTIFACT	ROLE
Platform	junit-platform-*	Launch, discover, report
Jupiter	junit-jupiter	Your test API (JUnit 5/6)
Vintage	junit-vintage-engine	Run JUnit 3/4 tests (deprecated in 6)
BOM	org.junit:junit-bom	Aligns versions

JUNIT 5 → 6

CHANGE	DETAIL
Java baseline	8 → 17
Versions	Platform, Jupiter, Vintage all 6.x
Vintage	Deprecated
Nullness	JSpecify annotations on the API
Removed	APIs deprecated in 5.x
Kotlin	suspend test methods supported

⚠ COMMON MISTAKES

- Mixing `org.junit.Test` (JUnit 4) and `org.junit.jupiter.api.Test` imports; the wrong one runs on the wrong engine or not at all
- Swapping expected and actual in `assertEquals`, which makes failure messages misleading
- Sharing state in instance fields expecting it to survive between tests

✓ TIPS & BEST PRACTICES

- Let the IDE generate the test class (⇧⌘T / Ctrl+Shift+T in IntelliJ)
- Name methods after behavior: `rejectsZeroQuantity`
- Check imports first when a test "doesn't run"

QUICK SUMMARY

- Platform — launches tests
- Jupiter — `@Test`, `Assertions`
- Vintage — JUnit 4 (deprecated in 6)
- JUnit 6 — Java 17+, one version

Setup with Maven & Gradle

ADD JUNIT TO THE BUILD

Import the **junit-bom** so every JUnit module has the same version, then depend on **junit-jupiter** (API, params and engine in one) in test scope. Maven's **Surefire** plugin (3.x) finds the JUnit Platform automatically; Gradle needs **useJUnitPlatform()** and the **junit-platform-launcher** on the test runtime classpath. The Maven and Gradle sets cover the build tools.

NOTES

- Spring Boot's starter already brings JUnit; don't add versions by hand
- Add `assertj-core` and `mockito-junit-jupiter` the same way
- Failsafe runs `*IT.java` integration tests in Maven's `verify` phase
- Test sources: `src/test/java`, resources: `src/test/resources`

IN SPRING BOOT

- `spring-boot-starter-test` includes JUnit Jupiter, AssertJ, Mockito, Hamcrest and JSONassert
- Boot's dependency management picks the JUnit version; override with the `junit-jupiter.version` property only if needed

POM.XML AND BUILD.GRADLE.KTS

```
<!-- pom.xml -->
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.junit</groupId>
      <artifactId>junit-bom</artifactId>
      <version>6.1.1</version>
      <type>pom</type><scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
<!-- Surefire 3.x detects the JUnit Platform -->

// build.gradle.kts
dependencies {
    testImplementation(platform("org.junit:junit-bom:6.1.1"))
    testImplementation("org.junit.jupiter:junit-jupiter")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
}
tasks.test { useJUnitPlatform() }
```

CODE EXPLAINED

- → The BOM sets one version; individual JUnit dependencies then omit `<version>`
- → `junit-jupiter` is an aggregate of the API, `-params` and `-engine` artifacts
- → Gradle runs nothing until `useJUnitPlatform()` is set
- → Gradle versions 9+ want the launcher declared explicitly

JUNIT 4 / TESTNG EQUIVALENT

- ↺ JUnit 4 `junit:junit:4.13.2` dependency; Surefire uses its JUnit 4 provider
- ↺ TestNG `org.testng:testng` dependency; Gradle `useTestNG()`

PREFER / AVOID

- ⇄ hard-coded versions on each JUnit artifact → the BOM
- ⇄ old Surefire 2.x → Surefire 3.x
- ⇄ separate `-api` + `-engine` dependencies → `junit-jupiter`

ARTIFACTS

ARTIFACT	CONTAINS
junit-jupiter	API + params + engine
junit-jupiter-params	Parameterized tests
junit-platform-launcher	Launcher (Gradle, tools)
junit-platform-suite	@Suite support
junit-vintage-engine	JUnit 4 tests
assertj-core	Fluent assertions
mockito-junit-jupiter	MockitoExtension

BUILD PLUGINS

TOOL	RUNS
Maven Surefire 3.x	Unit tests in `test` phase (*Test.java)
Maven Failsafe 3.x	Integration tests in `verify` (*IT.java)
Gradle test task	useJUnitPlatform()
Gradle JVM Test Suites	Separate integrationTest suite

⚠ COMMON MISTAKES

- Forgetting `useJUnitPlatform()` in Gradle; the build passes with zero tests run
- Mixing BOM and explicit versions so modules drift apart
- Adding JUnit versions manually in a Spring Boot project and fighting Boot's managed versions

✓ TIPS & BEST PRACTICES

- Check the test count in build output after setup; zero means misconfiguration
- Use Failsafe (Maven) or a separate Gradle source set for slow integration tests
- Let Spring Boot manage test library versions

QUICK SUMMARY

- `junit-bom` — one version
- `junit-jupiter` — everything you need
- Surefire 3 — auto-detects
- `useJUnitPlatform()` — required in Gradle

Running Tests

FROM THE IDE, THE BUILD OR THE CONSOLE

Run one test from the **IDE gutter**, a class or package from the build tool, or the whole suite in CI. Maven filters with `-Dtest=Class#method`, Gradle with `--tests`. The **ConsoleLauncher** runs tests without a build tool. **`junit-platform.properties`** in `src/test/resources` holds settings such as parallel execution and default display names.

NOTES

- IntelliJ: ^⇧R / Ctrl+Shift+F10 runs the test at the cursor
- Patterns allow `*`: `-Dtest="Cart*Test"`
- Gradle caches test results; `--rerun` forces a run
- Tag filters (`@Tag`) are sheet 06

COMMAND LINE

```
# Maven
mvn test                                  # all unit tests
mvn test -Dtest=CartTest                  # one class
mvn test -Dtest=CartTest#addsNewItem      # one method
mvn test -Dtest="Cart*Test,Order*Test"    # patterns
mvn verify                                # + Failsafe *IT.java tests
mvn test -Dgroups=fast -DexcludedGroups=slow

# Gradle
./gradlew test
./gradlew test --tests "shop.CartTest"
./gradlew test --tests "shop.CartTest.addsNewItem"
./gradlew test --tests "*Cart*" --rerun
./gradlew test --continuous               # rerun on change

# ConsoleLauncher (no build tool)
java -jar junit-platform-console-standalone.jar execute \
  --class-path target/test-classes:target/classes --scan-classpath
```

CODE EXPLAINED

- → `#method` selects one method in Maven; Gradle uses a dot
- → `mvn verify` runs integration tests after packaging
- → `-Dgroups` / `-DexcludedGroups` filter by `@Tag`
- → `--continuous` keeps Gradle watching for changes

JUNIT 4 / TESTNG EQUIVALENT

- ↺ **JUnit 4** Same Maven / Gradle commands; JUnit 4 classes picked by Surefire's JUnit 4 provider
- ↺ **TestNG** `testng.xml` suite files; `-Dsurefire.suiteXmlFiles=testng.xml`

PREFER / AVOID

- ⇄ running the whole suite for one change → IDE gutter or `-Dtest`
- ⇄ integration tests mixed with unit tests → Failsafe `*IT` or a Gradle source set
- ⇄ flags repeated on every run → `junit-platform.properties`

JUNIT-PLATFORM.PROPERTIES

KEY	EXAMPLE
junit.jupiter.displayname.generator.default	…ReplaceUnderscores
junit.jupiter.execution.parallel.enabled	true (sheet 16)
junit.jupiter.testinstance.lifecycle.default	per_class
junit.jupiter.execution.timeout.default	5 s
junit.jupiter.extensions.autodetection.enabled	true

WHERE RESULTS GO

TOOL	FOLDER
Maven Surefire	target/surefire-reports/
Maven Failsafe	target/failsafe-reports/
Gradle	build/test-results/test/ + build/reports/tests/
IntelliJ	Run tool window

⚠ COMMON MISTAKES

- Quoting patterns wrong in the shell so `*` expands to file names
- Expecting `mvn test` to run `*IT` classes (Failsafe runs those in `verify`)
- Thinking tests ran when Gradle reported "UP-TO-DATE"

✓ TIPS & BEST PRACTICES

- Use the IDE for the edit-test loop and the build tool for the full run
- Open `build/reports/tests/test/index.html` for a readable Gradle report
- Use `--rerun` (Gradle 7.6+) instead of `cleanTest`

QUICK SUMMARY

- IDE gutter — one test
- `-Dtest=Class#method` — Maven
- `--tests "pkg.Class.method"` — Gradle
- `junit-platform.properties` — shared settings

Sheet 04

Test Classes & Methods

@Test · package-private visibility · naming conventions · @DisplayName · @DisplayNameGeneration · test class layout

READABLE TESTS AND READABLE REPORTS

A test class is a plain class with `@Test` methods; neither needs to be `public`. Name methods after behavior, and use **`@DisplayName`** for a sentence in reports, or **`@DisplayNameGeneration`** to turn `snake_case_names` into words automatically. Arrange-Act-Assert inside each method keeps the steps easy to see.

NOTES

- Test methods must not be `private` or `static`, and return `void`
- Display names show in IDE and build reports, not in code
- Emoji and spaces are allowed in `@DisplayName`
- `@IndicativeSentencesGeneration` builds sentences from class + method

NAMES AND LAYOUT

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Cart")
@DisplayNameGeneration(DisplayNameGenerator.ReplaceUnderscores.class)
class CartTest {

    @Test
    void total_is_zero_when_empty() {              // → "total is zero when empty"
        assertEquals(0, new Cart().total());
    }

    @Test
    @DisplayName("applies a 25% discount")
    void discount() {
        // arrange
        Cart cart = new Cart();
        cart.add("A1", 2, 10.00);
        // act
        double total = cart.total(0.25);
        // assert
        assertEquals(15.00, total, 0.001);
    }
}
```

CODE EXPLAINED

- → `ReplaceUnderscores` turns method names into readable report lines
- → `@DisplayName` on the class prefixes every test in the report
- → Comments for arrange / act / assert are optional; blank lines often suffice
- → The third argument to `assertEquals` for doubles is the tolerance

JUNIT 4 / TESTNG EQUIVALENT

- ↺ **JUnit 4** No display names; method names only; classes and methods must be `public`
- ↺ **TestNG** `@Test(description = "…")` and `testName` in reports

PREFER / AVOID

- ⇄ `test1`, `test2` → names that describe behavior
- ⇄ `public` everywhere → package-private
- ⇄ long setup inside each test → `@BeforeEach` or a factory method

NAMING STYLES

STYLE	EXAMPLE
`camelCase behavior`	rejectsZeroQuantity
`snake_case + ReplaceUnderscores`	rejects_zero_quantity
`should…When…`	shouldRejectWhenQtyIsZero
`Given / when / then`	givenEmptyCart_whenTotal_thenZero

DISPLAY NAME GENERATORS

GENERATOR	RESULT
`Standard (default)`	method name + ()
`Simple`	method name without ()
`ReplaceUnderscores`	underscores → spaces
`IndicativeSentences`	Class, method as a sentence

⚠ COMMON MISTAKES

- Making test methods `private` or `static`; they're silently skipped
- Testing several behaviors in one method named `testCart`
- Display names that contradict what the test checks after a refactor

✓ TIPS & BEST PRACTICES

- Set a default generator in `junit-platform.properties` for the whole project
- Keep one Act per test
- Group related tests with `@Nested` (sheet 06) instead of long names

QUICK SUMMARY

- `@Test` — package-private `void` method
- `@DisplayName` — sentence in reports
- `@DisplayNameGeneration` — names from method names
- Arrange-Act-Assert

Lifecycle

@BeforeEach / @AfterEach · @BeforeAll / @AfterAll · static requirement · @TestInstance(PER_CLASS) · execution order

SETUP AND TEARDOWN AROUND TESTS

JUnit creates a **new instance** of the test class for every test method, then calls @BeforeEach, the test, and @AfterEach. @BeforeAll and @AfterAll run once per class and must be **static**, unless the class uses **@TestInstance(Lifecycle.PER_CLASS)**, which keeps one instance for all tests.

NOTES

- `@AfterEach` runs even if the test failed
- Superclass `@BeforeEach` runs before the subclass's
- Parameters like `TestInfo` can be injected into lifecycle methods
- Avoid relying on test order; there is none by default

IN SPRING BOOT

- `@SpringBootTest` contexts are cached and reused across test classes with the same configuration
- `@DirtiesContext` throws the cached context away, which slows the suite
- `@BeforeAll` is a good place for Testcontainers, not for Spring beans (not injected yet in static methods)

LIFECYCLE METHODS

```
import org.junit.jupiter.api.*;

class OrderRepositoryTest {
    static Database db;
    OrderRepository repo;

    @BeforeAll
    static void startDb() {           // once, before all tests (static)
        db = Database.startInMemory();
    }

    @BeforeEach
    void setUp() {                    // before every test
        repo = new OrderRepository(db);
        db.beginTransaction();
    }

    @Test
    void savesOrder() {
        repo.save(new Order(1, 10.0));
        assertEquals(1, repo.count());
    }

    @AfterEach
    void rollback() { db.rollback(); }     // after every test, pass or fail

    @AfterAll
    static void stopDb() { db.stop(); }
}

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class SharedInstanceTest {
    @BeforeAll void init() { }        // no static needed
}
```

CODE EXPLAINED

- → Order per test: constructor → `@BeforeEach` → test → `@AfterEach`
- → Rolling back in `@AfterEach` keeps tests independent while sharing one database
- → `PER_CLASS` is handy in Kotlin, where static methods are awkward
- → Instance fields reset each test because each test gets a new instance

JUNIT 4 / TESTNG EQUIVALENT

- JUnit 4 `@Before` / `@After` / `@BeforeClass` / `@AfterClass` (static)
- TestNG `@BeforeMethod` / `@AfterMethod` / `@BeforeClass` / `@AfterClass` (not static), plus `@BeforeSuite`

PREFER / AVOID

- ⇄ static mutable fields shared by tests → per-test setup
- ⇄ heavy `@BeforeEach` work → `@BeforeAll` + reset per test
- ⇄ cleanup inside each test → `@AfterEach`

EXECUTION ORDER

STEP	RUNS
1. `@BeforeAll`	Once per class
2. `constructor`	Per test (PER_METHOD)
3. `@BeforeEach`	Superclass first, then subclass
4. `@Test`	The test
5. `@AfterEach`	Subclass first, then superclass
6. `@AfterAll`	Once per class

ANNOTATIONS

ANNOTATION	STATIC?
`@BeforeAll / @AfterAll`	Yes (unless PER_CLASS)
`@BeforeEach / @AfterEach`	No
`@TestInstance(PER_CLASS)`	One instance per class
`@TestInstance(PER_METHOD)`	Default: new instance per test

⚠ COMMON MISTAKES

- Non-static `@BeforeAll` without `PER_CLASS` (JUnit reports an error)
- Expecting instance fields set in one test to be there in the next
- Using `PER_CLASS` and leaking state between tests through fields

✓ TIPS & BEST PRACTICES

- Prefer the default per-method lifecycle for isolation
- Use `PER_CLASS` in Kotlin or when `@BeforeAll` needs instance state
- Keep teardown in `@AfterEach` so it runs on failure

QUICK SUMMARY

- `@BeforeEach` / `@AfterEach` — every test
- `@BeforeAll` / `@AfterAll` — once, static
- new instance per test (default)
- `PER_CLASS` — one instance

Sheet 06

Nested, Tagged & Ordered Tests

@Nested · @Tag · @Disabled · @TestMethodOrder · @Order · @Suite · tag filtering in Maven / Gradle

GROUP, LABEL AND SELECT

@Nested inner classes group tests by situation ("when empty", "after adding an item"), each with its own setup that builds on the outer class. **@Tag** labels tests so builds can include or exclude them (`fast`, `integration`). **@Disabled** skips with a reason. **@TestMethodOrder** fixes an order when you truly need one, and **@Suite** groups classes into a named run.

NOTES

- `@Nested` classes must be non-static inner classes
- Outer `@BeforeEach` runs before inner `@BeforeEach`
- Tag expressions: `fast & !slow`, `integration | e2e`
- `@Suite` needs `junit-platform-suite`

IN SPRING BOOT

- `@Nested` test classes reuse the outer class's `@SpringBootTest` configuration and context
- `@ActiveProfiles` on an inner class can switch profiles for that group

NESTED AND TAGGED

```
@DisplayName("Cart")
class CartTest {
    Cart cart;

    @BeforeEach void newCart() { cart = new Cart(); }

    @Nested
    class WhenEmpty {
        @Test void totalIsZero() { assertEquals(0, cart.total()); }
    }

    @Nested
    class AfterAddingAnItem {
        @BeforeEach void addOne() { cart.add("A1", 1, 5.00); }

        @Test void countIsOne() { assertEquals(1, cart.size()); }
        @Test @Tag("slow") void syncsToServer() { /* … */ }
    }

    @Test
    @Disabled("flaky until #142 is fixed")
    void legacyPricing() { }
}

@Suite
@SelectPackages("shop.checkout")
@IncludeTags("integration")
class CheckoutIntegrationSuite { }

// Maven:  mvn test -Dgroups="fast & !slow"
// Gradle: tasks.test { useJUnitPlatform { includeTags("fast"); excludeTags("slow") } }
```

CODE EXPLAINED

→ Each `@Nested` class reads like a scenario in the report

→ `AfterAddingAnItem` gets `newCart()` then `addOne()` before each test

→ `@Disabled` always takes a reason

→ A `@Suite` class selects tests by package, class or tag

JUNIT 4 / TESTNG EQUIVALENT

JUnit 4 No `@Nested` (use `Enclosed` runner); `@Category` for tags; `@Ignore`; `@FixMethodOrder`; `@RunWith(Suite.class)`

TestNG `@Test(groups = "fast")`, `enabled = false`, `priority` / `dependsOnMethods`, `testng.xml` suites

PREFER / AVOID

⇄ long test classes with name prefixes → `@Nested` groups

⇄ commented-out tests → `@Disabled("reason + issue")`

⇄ tests that depend on order → independent tests (use `@Order` only for genuine sequences)

ANNOTATIONS

ANNOTATION	EFFECT
`@Nested`	Inner class group
`@Tag("x")`	Label for filtering
`@Disabled("why")`	Skip
`@TestMethodOrder(…)`	Method order
`@Order(n)`	Position with OrderAnnotation
`@Suite + @Select…`	Named group of tests

TAG EXPRESSIONS

EXPRESSION	SELECTS
`fast`	Tagged fast
`!slow`	Not slow
`fast & !slow`	Both conditions
`integration | e2e`	Either
`(a | b) & !c`	Grouped

⚠ COMMON MISTAKES

- Making `@Nested` classes `static`; they won't be discovered
- Using `@Order` to make tests depend on each other
- `@Disabled` tests piling up with no issue link

✓ TIPS & BEST PRACTICES

- Use constants for tag names to avoid typos
- Run `fast` tests locally and everything in CI
- Review `@Disabled` tests regularly

QUICK SUMMARY

- `@Nested` — scenario groups
- `@Tag` + build filters — select
- `@Disabled("why")` — skip
- `@Suite` — named group

Core Assertions

assertEquals · assertTrue / assertFalse · assertNull · assertSame · assertArrayEquals · assertIterableEquals · message suppliers

THE BUILT-IN ASSERTIONS CLASS

`org.junit.jupiter.api.Assertions` holds static methods you import with `import static …Assertions.*`. Each takes the **expected** value first, the **actual** second, and an optional **message** last, either a `String` or a `Supplier<String>` that is only built if the assertion fails. For collections, `assertIterableEquals` compares element by element.

NOTES

- `assertEquals(double, double, delta)` for floating point
- `assertSame` checks identity (`==`); `assertEquals` uses `equals()`
- `assertInstanceOf` returns the cast value
- `fail("msg")` fails unconditionally

ASSERTIONS IN USE

```
import static org.junit.jupiter.api.Assertions.*;

@Test
void assertions() {
    Order order = service.place(cart);

    assertEquals(OrderStatus.NEW, order.status());
    assertEquals(19.99, order.total(), 0.001);                // delta for doubles
    assertNotEquals(0, order.id());
    assertTrue(order.lines().size() > 0, "order has lines");
    assertFalse(order.isPaid());
    assertNull(order.paidAt());
    assertNotNull(order.createdAt(), () -> "createdAt missing on " + order);
    assertSame(cart.customer(), order.customer());
    assertArrayEquals(new int[]{1, 2}, order.lineIds());
    assertIterableEquals(List.of("A1", "B2"), order.skus());
    assertLinesMatch(List.of("Order \\d+", ">> >>", "Total: .*"), order.receiptLines());
    Invoice inv = assertInstanceOf(Invoice.class, order.document());
}
```

CODE EXPLAINED

- → The message comes **last** in JUnit 5/6 (it was first in JUnit 4)
- → The lambda message is built only when the assertion fails
- → `assertLinesMatch` allows regex lines and `>> >>` to skip lines
- → `assertInstanceOf` both checks the type and returns the cast object

JUNIT 4 / TESTNG EQUIVALENT

- ↺ JUnit 4 `Assert.assertEquals("msg", expected, actual)` (message first); `assertThat` with Hamcrest
- ↺ TestNG `Assert.assertEquals(actual, expected, "msg")` — actual first

PREFER / AVOID

- ⇄ `assertTrue(a.equals(b))` → `assertEquals(b, a)` (better message)
- ⇄ `assertTrue(list.size() == 2)` → AssertJ `hasSize(2)` (sheet 09)
- ⇄ string messages built with `+` on every call → `Supplier<String>`

ASSERTIONS

METHOD	CHECKS
`assertEquals(e, a[, delta])`	equals() / within delta
`assertNotEquals`	Not equal
`assertTrue / assertFalse`	Boolean
`assertNull / assertNotNull`	Null
`assertSame / assertNotSame`	Identity
`assertArrayEquals`	Arrays
`assertIterableEquals`	Iterables, deep
`assertLinesMatch`	Lines with regex

ARGUMENT ORDER

LIBRARY	ORDER
`JUnit 5/6`	expected, actual, message
`JUnit 4`	message, expected, actual
`TestNG`	actual, expected, message
`AssertJ`	assertThat(actual).isEqualTo(expected)

⚠ COMMON MISTAKES

- Swapping expected and actual, so failures say "expected <bad> but was <good>"
- Comparing doubles without a delta
- Using `assertSame` on strings or boxed numbers

✓ TIPS & BEST PRACTICES

- Use AssertJ for collections and complex objects (sheet 09)
- Give a message when the values alone don't explain the failure
- Static-import `Assertions.*` once per file

QUICK SUMMARY

- `assertEquals(expected, actual)`
- message last (String or Supplier)
- delta for doubles
- `assertInstanceOf` — check and cast

Sheet 08

Exceptions, Grouped Assertions & Timeouts

assertThrows · assertThrowsExactly · assertDoesNotThrow · assertAll · assertTimeout · assertTimeoutPreemptively · @Timeout

ERRORS, SEVERAL CHECKS AT ONCE, AND TIME

assertThrows passes if the lambda throws the given type (or a subclass) and returns the exception so you can check its message. **assertAll** runs several assertions and reports every failure together instead of stopping at the first. **assertTimeout** and **@Timeout** fail tests that run too long.

NOTES

- `assertThrowsExactly` rejects subclasses
- `assertAll` takes `Executable` lambdas, can be nested
- `assertTimeoutPreemptively` runs code in another thread and aborts it
- `@Timeout` works on methods, classes, and lifecycle methods

THROWS, GROUPS, TIME

```
@Test
void rejectsNegativeQuantity() {
    var ex = assertThrows(IllegalArgumentException.class,
                          () -> cart.add("A1", -1));
    assertEquals("qty must be > 0", ex.getMessage());
}

@Test
void exactType() {
    assertThrowsExactly(OrderNotFoundException.class, () -> service.find(99));
}

@Test
void doesNotThrow() {
    Order o = assertDoesNotThrow(() -> service.find(1));   // returns the value
}

@Test
void addressFields() {
    Address a = customer.address();
    assertAll("address",
        () -> assertEquals("1 Main St", a.street()),
        () -> assertEquals("Tripoli", a.city()),
        () -> assertEquals("IA", a.state()));
}

@Test
void fastEnough() {
    assertTimeout(Duration.ofMillis(200), () -> pricing.quote(cart));
}

@Test
@Timeout(value = 2, unit = TimeUnit.SECONDS)
void finishesInTwoSeconds() { importer.run(); }
```

CODE EXPLAINED

→ The exception returned by `assertThrows` lets you assert on message or fields

→ All three address checks run, and every failure is reported

→ `assertTimeout` lets the code finish, then fails if it was slow; the preemptive version aborts it

→ Set a project-wide default timeout in `junit-platform.properties`

JUNIT 4 / TESTNG EQUIVALENT

↺ **JUnit 4** `@Test(expected = X.class)`, `ExpectedException` rule; `@Test(timeout = 2000)`

↺ **TestNG** `@Test(expectedExceptions = X.class, timeOut = 2000)`; `SoftAssert` for grouped checks

PREFER / AVOID

⇄ `try { …; fail(); } catch (X e) {}` → `assertThrows`

⇄ several asserts where you want all failures → `assertAll`

⇄ catching `Exception.class` → the specific type

EXCEPTION ASSERTIONS

METHOD	PASSES WHEN
`assertThrows(T, exec)`	Throws T or subclass; returns it
`assertThrowsExactly(T, exec)`	Throws exactly T
`assertDoesNotThrow(exec)`	Nothing thrown; returns result
`assertAll(heading, execs…)`	All pass (reports all failures)

TIMEOUTS

FORM	BEHAVIOR
`assertTimeout(d, exec)`	Runs to completion, then checks
`assertTimeoutPreemptively(d, exec)`	Aborts at d (other thread)
`@Timeout(n)`	Fails test after n s
`junit.jupiter.execution.timeout.default`	Project default

⚠ COMMON MISTAKES

- Wrapping several statements in the `assertThrows` lambda so the wrong one throws
- `assertTimeoutPreemptively` with code that uses thread-locals (Spring transactions)
- Catching `Exception` and passing on a typo-caused `NullPointerException`

✓ TIPS & BEST PRACTICES

- Keep only the call that should throw inside the lambda
- Use `assertAll` for checking the fields of one object
- Prefer `@Timeout` for slow-test guards; it's declarative

QUICK SUMMARY

- `assertThrows` — returns the exception
- `assertAll` — report every failure
- `assertTimeout` / preemptive
- `@Timeout` — declarative limit

AssertJ

assertThat · isEqualTo · containsExactly · extracting · satisfies · assertThatThrownBy · SoftAssertions

FLUENT ASSERTIONS THAT READ LIKE SENTENCES

AssertJ is the most popular assertion library for Java. `assertThat(actual)` returns a type-specific object whose methods your IDE autocompletes: strings get `startsWith`, lists get `containsExactly`, optionals get `isPresent`. Failure messages are detailed, and `extracting` pulls fields out of objects for compact collection checks. Spring Boot includes it.

NOTES

- `import static org.assertj.core.api.Assertions.*`
- Chain several checks on one value
- `usingRecursiveComparison()` compares objects field by field
- Soft assertions collect all failures, like `assertAll`

IN SPRING BOOT

- `MockMvcTester` returns AssertJ-style assertions for HTTP responses (`assertThat(mvc.get().uri("/x")).hasStatusOk()`)
- `JsonContent` in `@JsonTest` supports `extractingJsonPathStringValue`
- Boot's `ApplicationContextRunner` has AssertJ assertions for auto-configuration

FLUENT CHECKS

```java
import static org.assertj.core.api.Assertions.*;

@Test
void fluent() {
    assertThat(order.total()).isEqualTo(19.99).isPositive();
    assertThat(order.total()).isCloseTo(19.99, within(0.001));
    assertThat(order.reference()).startsWith("ORD-").hasSize(10);
    assertThat(order.lines())
        .hasSize(2)
        .extracting(Line::sku, Line::qty)
        .containsExactly(tuple("A1", 2), tuple("B2", 1));
    assertThat(order.tags()).containsExactlyInAnyOrder("gift", "rush");
    assertThat(order.paidAt()).isEmpty();                    // Optional
    assertThat(order).usingRecursiveComparison()
        .ignoringFields("id", "createdAt")
        .isEqualTo(expectedOrder);
    assertThat(order.customer()).satisfies(c -> {
        assertThat(c.name()).isEqualTo("Ann");
        assertThat(c.email()).contains("@");
    });
}

@Test
void exceptions() {
    assertThatThrownBy(() -> cart.add("A1", 0))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("qty");
    assertThatExceptionOfType(OrderNotFoundException.class)
        .isThrownBy(() -> service.find(99));
}
```

CODE EXPLAINED

- `extracting` + `tuple` checks several fields of every element in one line
- `usingRecursiveComparison` with `ignoringFields` skips generated values
- `satisfies` groups checks about one nested object

JUNIT 4 / TESTNG EQUIVALENT

- **JUnit 4** Hamcrest `assertThat(x, is(equalTo(y)))` ; AssertJ works with JUnit 4 too
- **TestNG** TestNG users also use AssertJ; `SoftAssert` is TestNG's built-in version

PREFER / AVOID

- `assertEquals(2, list.size())` → `assertThat(list).hasSize(2)`
- loops of asserts over a list → `extracting(...).containsExactly(...)`
- overriding `equals` just for tests → `usingRecursiveComparison()`

COMMON METHODS

TYPE	METHODS
`Any`	isEqualTo · isNotNull · isInstanceOf · isIn
`Number`	isPositive · isBetween · isCloseTo(v, within(d))
`String`	startsWith · contains · matches · isBlank
`Collection`	hasSize · contains · containsExactly · containsOnly · isEmpty
`Optional`	isPresent · isEmpty · contains · hasValue
`Map`	containsEntry · containsKeys · hasSize

EXCEPTION FORMS

FORM	USE
`assertThatThrownBy(…)`	Any throwable, then chain
`assertThatExceptionOfType(T).isThrownBy(…)`	Type first
`assertThatNoException().isThrownBy(…)`	Nothing thrown
`assertThatIllegalArgumentException()`	Shortcut for common types

⚠ COMMON MISTAKES

- Forgetting to call a terminal check: `assertThat(x);` alone asserts nothing
- Mixing JUnit's `assertEquals` and AssertJ in confusing ways in one test
- Using `contains` when order matters (`containsExactly`)

✓ TIPS & BEST PRACTICES

- Let autocomplete guide you after `assertThat(…).`
- Use `as("description")` before a check to label it in failures
- Turn on AssertJ's IntelliJ / Sonar rules to catch assertions with no check

QUICK SUMMARY

- `assertThat(actual)` — fluent, typed
- `extracting` + `containsExactly` — collections
- `usingRecursiveComparison` — objects
- `assertThatThrownBy` / soft assertions

Assumptions & Conditional Tests

RUN ONLY WHERE IT MAKES SENSE

An **assumption** aborts a test (reported as skipped, not failed) when a precondition isn't met, such as a missing environment variable. **Conditional annotations** decide before the test starts: only on Linux, only on Java 21+, only when `CI=true`, or when a custom method returns true. Both keep the suite green on machines that can't run every test.

NOTES

- A failed assumption marks the test aborted / skipped
- `assumingThat` runs part of a test only when the condition holds
- Every `@Enabled…` has a `@Disabled…` twin
- `disabledReason` shows why in reports

ASSUMPTIONS AND CONDITIONS

```
import static org.junit.jupiter.api.Assumptions.*;
import org.junit.jupiter.api.condition.*;

@Test
void onlyOnCi() {
    assumeTrue("true".equals(System.getenv("CI")), "CI only");
    // …
}

@Test
void partly() {
    assumingThat(System.getenv("STRIPE_KEY") != null,
        () -> assertTrue(payments.liveCheck()));
    assertTrue(payments.offlineCheck());       // always runs
}

@Test @EnabledOnOs(OS.LINUX)
void usesProcFs() { }

@Test @EnabledForJreRange(min = JRE.JAVA_21)
void usesVirtualThreads() { }

@Test @EnabledIfEnvironmentVariable(named = "DB_URL", matches = "jdbc:postgresql:.*")
void againstPostgres() { }

@Test @DisabledIfSystemProperty(named = "skipSlow", matches = "true",
                                disabledReason = "slow")
void slowReport() { }

@Test @EnabledIf("dockerAvailable")
void withDocker() { }
static boolean dockerAvailable() { return DockerClientFactory.instance().isDockerAvailable(); }
```

CODE EXPLAINED

- → `assumeTrue` aborts the rest of the test when false
- → `assumingThat` guards only the lambda; the rest of the test still runs
- → `matches` takes a regex
- → `@EnabledIf` names a static method (or `Class#method`) returning boolean

JUNIT 4 / TESTNG EQUIVALENT

- ↺ JUnit 4 `Assume.assumeTrue(...)`; no conditional annotations (use rules)
- ↺ TestNG `throw new SkipException("…")`; `enabled = false`; group-based filtering

PREFER / AVOID

- ⇄ `if (!condition) return;` (reports a pass) → `assumeTrue` (reports skipped)
- ⇄ OS checks inside tests → `@EnabledOnOs`
- ⇄ commented-out platform tests → conditional annotations

CONDITION ANNOTATIONS

ANNOTATION	CONDITION
`@EnabledOnOs / @DisabledOnOs`	OS.LINUX, MAC, WINDOWS…
`@EnabledOnJre / @EnabledForJreRange`	Java version
`@EnabledIfSystemProperty`	System property regex
`@EnabledIfEnvironmentVariable`	Env var regex
`@EnabledIf / @DisabledIf`	Custom method
`@EnabledInNativeImage`	GraalVM native image

ASSUMPTIONS

METHOD	EFFECT
`assumeTrue(cond[, msg])`	Abort if false
`assumeFalse(cond)`	Abort if true
`assumingThat(cond, exec)`	Run exec only if true
`abort(msg)`	Abort unconditionally (5.9+)

⚠ COMMON MISTAKES

- Returning early from a test instead of assuming, which reports a false pass
- Conditions that skip tests in CI by accident (wrong env var name)
- Hiding broken tests behind assumptions

✓ TIPS & BEST PRACTICES

- Check skipped counts in CI reports; a sudden jump means a condition broke
- Use `@EnabledIf` with Testcontainers' Docker check
- Always give `disabledReason`

QUICK SUMMARY

- `assumeTrue` — skip at run time
- `assumingThat` — guard part of a test
- `@EnabledOnOs` / `@EnabledForJreRange` — before start
- `@EnabledIf` — custom condition

Sheet 11

Parameterized Tests — Sources

@ParameterizedTest · @ValueSource · @CsvSource · @CsvFileSource · @EnumSource · @NullAndEmptySource · name = patterns

ONE METHOD, MANY INPUTS

@ParameterizedTest runs a method once per set of arguments supplied by a **source** annotation: literal values, CSV rows, a CSV file, enum constants, or null and empty values. JUnit converts strings to the parameter types (numbers, enums, dates). Each invocation appears separately in the report, named by the `name` pattern.

NOTES

- Needs `junit-jupiter-params` (included in `junit-jupiter`)
- Combine sources: `@NullAndEmptySource` + `@ValueSource`
- Text blocks make `@CsvSource` tables readable
- `@ParameterizedClass` (5.13+) parameterizes a whole class

IN SPRING BOOT

- Parameterized tests work in `@SpringBootTest` classes; the context is created once and reused for every invocation
- Injected beans (`@Autowired`) and parameters can be mixed: sources first, then resolved parameters

SOURCES

```
@ParameterizedTest
@ValueSource(strings = {"A1", "B22", "C333"})
void validSkus(String sku) { assertTrue(Sku.isValid(sku)); }

@ParameterizedTest(name = "{0} + {1} = {2}")
@CsvSource({
    "1, 2, 3",
    "10, -4, 6",
    "'1,000', 1, 1001",            // quotes protect the comma
})
void adds(String a, int b, int expected) { /* … */ }

@ParameterizedTest
@CsvSource(useHeadersInDisplayName = true, textBlock = """
    country, rate
    US,      0.07
    DE,      0.19
    """)
void taxRates(String country, double rate) {
    assertEquals(rate, Tax.rate(country), 1e-9);
}

@ParameterizedTest
@CsvFileSource(resources = "/prices.csv", numLinesToSkip = 1)
void fromFile(String sku, BigDecimal price) { }

@ParameterizedTest
@EnumSource(value = Status.class, names = {"PAID", "SHIPPED"})
void finalStates(Status s) { assertTrue(s.isFinal()); }

@ParameterizedTest
@NullAndEmptySource
@ValueSource(strings = {"  ", "\t"})
void blankNames(String name) { assertThrows(IllegalArgumentException.class, () -> new User(name)); }
```

CODE EXPLAINED

- `{0}`, `{1}` in `name` insert arguments; `{index}` the invocation number
- Strings convert to `int`, `double`, `BigDecimal`, enums and `java.time` types automatically
- `textBlock` with headers reads like a table
- `@NullAndEmptySource` adds `null` and `""` to the other values

JUNIT 4 / TESTNG EQUIVALENT

- JUnit 4 `@RunWith(Parameterized.class)` with a constructor and `@Parameters` method (class-level)
- TestNG `@DataProvider` method + `@Test(dataProvider = "…")`

PREFER / AVOID

- loops of asserts over cases → `@ParameterizedTest`
- copy-pasted tests differing by one value → one parameterized test
- unreadable report names → `name = "…"` pattern

SOURCE ANNOTATIONS

SOURCE	SUPPLIES
`@ValueSource`	Literals of one type
`@CsvSource`	Inline CSV rows
`@CsvFileSource`	CSV file rows
`@EnumSource`	Enum constants
`@NullSource` / `@EmptySource`	null / empty
`@NullAndEmptySource`	Both
`@MethodSource` / `@FieldSource`	Code (sheet 12)

NAME PLACEHOLDERS

PLACEHOLDER	INSERTS
`{index}`	Invocation number
`{0}, {1}…`	Argument
`{arguments}`	All arguments
`{argumentsWithNames}`	name=value pairs
`{displayName}`	Method display name

⚠ COMMON MISTAKES

- Unquoted commas in `@CsvSource` values splitting a value in two
- Forgetting that blank CSV cells become `null`
- Very long CSV tables in annotations; move them to `@CsvFileSource`

✓ TIPS & BEST PRACTICES

- Use `textBlock` CSV for anything with more than a few rows
- Give each invocation a readable `name`
- Put the expected value in each row

QUICK SUMMARY

- `@ParameterizedTest` + a source
- `@CsvSource` / `textBlock` — tables
- `@EnumSource` / `@NullAndEmptySource`
- `name = "…"` — readable reports

Sheet 12

Method Sources & Argument Providers

`@MethodSource` · `Stream<Arguments>` · `@FieldSource` · `ArgumentsProvider` · `ArgumentsAccessor` · `@AggregateWith` · converters

ARGUMENTS FROM CODE

When cases need real objects, supply them from code. **`@MethodSource`** names a static factory method returning a `Stream` of `Arguments`; **`@FieldSource`** reads a static field. A custom **`ArgumentsProvider`** can be shared across classes. On the receiving side, **`ArgumentsAccessor`** and **`@AggregateWith`** turn many arguments into one object.

NOTES

- Without a name, `@MethodSource` looks for a method with the test's name
- Factory methods are static unless `PER_CLASS`
- `Arguments.of(...)` / static `arguments(...)`
- External methods: `@MethodSource("shop.Cases#orders")`

CODE-SUPPLIED CASES

```
import static org.junit.jupiter.params.provider.Arguments.arguments;

@ParameterizedTest
@MethodSource("discountCases")
void discounts(Cart cart, double rate, double expected) {
    assertEquals(expected, cart.total(rate), 0.001);
}

static Stream<Arguments> discountCases() {
    return Stream.of(
        arguments(cartOf(10, 10), 0.25, 15.0),
        arguments(cartOf(), 0.50, 0.0),
        arguments(cartOf(99.99), 0.0, 99.99));
}

static final List<String> RESERVED = List.of("admin", "root");
@ParameterizedTest @FieldSource("RESERVED")
void reservedNames(String name) { assertFalse(Usernames.isAvailable(name)); }

@ParameterizedTest
@ArgumentsSource(CountryRatesProvider.class)
void rates(String country, double rate) { }

class CountryRatesProvider implements ArgumentsProvider {
    public Stream<? extends Arguments> provideArguments(ParameterDeclarations p, ExtensionContext c) {
        return Tax.table().entrySet().stream().map(e -> arguments(e.getKey(), e.getValue()));
    }
}

@ParameterizedTest
@CsvSource({"Ann, Smith, 34", "Bo, Li, 27"})
void people(@AggregateWith(PersonAggregator.class) Person p) { assertTrue(p.age() > 0); }
```

CODE EXPLAINED

- → Each `arguments(...)` becomes one invocation
- → `@FieldSource` (5.11+) reads a `List`, array or `Supplier` field
- → An `ArgumentsProvider` can compute cases from anywhere and be reused
- → `@AggregateWith` builds one parameter from several CSV columns

JUNIT 4 / TESTNG EQUIVALENT

- ↺ JUnit 4 `@Parameters` static method returning `Collection<Object[]>`
- ↺ TestNG `@DataProvider(name = "…")` returning `Object[][]` or an `Iterator`

PREFER / AVOID

- ⇄ `Object[][]` arrays → `Stream<Arguments>`
- ⇄ the same provider copied between classes → an `ArgumentsProvider` or external `@MethodSource`
- ⇄ eight parameters → `@AggregateWith` into a record

CODE SOURCES

SOURCE	READS
`@MethodSource("m")`	Static method returning Stream / Iterable / array
`@FieldSource("f")`	Static field (5.11+)
`@ArgumentsSource(P.class)`	Custom provider
`@MethodSource("pkg.Class#m")`	Method in another class

RECEIVING ARGUMENTS

TOOL	USE
`ArgumentsAccessor`	Read by index: getString(0), getInteger(1)
`@AggregateWith(A.class)`	Build one object from many
`@ConvertWith(C.class)`	Custom conversion for one parameter
`Implicit conversion`	String → number, enum, date, URI…

⚠ COMMON MISTAKES

- Non-static factory methods without `PER_CLASS`
- Building expensive objects for every case when only some need them
- Factory method names that drift from the annotation string (rename refactors miss them)

✓ TIPS & BEST PRACTICES

- Name factory methods after what they supply (`discountCases`)
- Use records with `@AggregateWith` for wide rows
- Keep shared case providers in a `test/…/Cases` class

QUICK SUMMARY

- `@MethodSource` — `Stream<Arguments>`
- `@FieldSource` — a static field
- `ArgumentsProvider` — reusable
- `@AggregateWith` — many columns → one object

Dynamic & Repeated Tests

TESTS BUILT AT RUN TIME, TESTS RUN MANY TIMES

A **@TestFactory** method returns a stream or collection of **DynamicTest** objects, so tests can be generated from data found at run time (files in a folder, rows from a service). **DynamicContainer** groups them into a tree. **@RepeatedTest** runs the same test N times, which helps find flaky behavior.

NOTES

- Dynamic tests don't get `@BeforeEach` / `@AfterEach` per dynamic test, only per factory
- Each dynamic test has a display name and an `Executable`
- `failureThreshold` stops repeating after N failures
- `RepetitionInfo` gives the current repetition number

FACTORIES AND REPETITION

```
import static org.junit.jupiter.api.DynamicTest.dynamicTest;
import static org.junit.jupiter.api.DynamicContainer.dynamicContainer;

@TestFactory
Stream<DynamicTest> everyFixtureFileParses() throws IOException {
    return Files.list(Path.of("src/test/resources/orders"))
        .map(path -> dynamicTest(path.getFileName().toString(),
                                 () -> assertNotNull(OrderParser.parse(path))));
}

@TestFactory
Stream<DynamicContainer> perCountry() {
    return Stream.of("US", "DE").map(c -> dynamicContainer(c, Stream.of(
        dynamicTest("has a rate", () -> assertTrue(Tax.rate(c) > 0)),
        dynamicTest("formats money", () -> assertNotNull(Money.format(1, c))))));
}

@RepeatedTest(value = 50, failureThreshold = 1,
              name = "{displayName} {currentRepetition}/{totalRepetitions}")
void idsAreUnique(RepetitionInfo info) {
    assertNotEquals(IdGen.next(), IdGen.next());
}
```

CODE EXPLAINED

→ File names become test names in the report

→ Containers show as folders in the IDE test tree

→ `failureThreshold = 1` stops after the first failure, saving time on a flake hunt

→ The repetition placeholders build readable names

JUNIT 4 / TESTNG EQUIVALENT

↺ **JUnit 4** No dynamic tests; use the `Parameterized` runner with computed parameters; no `@RepeatedTest` (loop or rule)

↺ **TestNG** `@Factory` creates test class instances; `invocationCount = 50` repeats

PREFER / AVOID

⇄ a loop of asserts over files → `@TestFactory` with one dynamic test per file

⇄ fixed cases known at compile time → `@ParameterizedTest`

⇄ rerunning a flaky test by hand → `@RepeatedTest` with `failureThreshold`

DYNAMIC API

TYPE / METHOD	CREATES
`dynamicTest(name, exec)`	One test
`dynamicContainer(name, stream)`	Group
`DynamicTest.stream(iter, nameFn, exec)`	Many from data
`@TestFactory return types`	Stream, Collection, Iterable, Iterator, array

@REPEATEDTEST

OPTION / PLACEHOLDER	MEANING
`value`	Number of repetitions
`failureThreshold`	Stop after N failures
`{currentRepetition}`	Current number
`{totalRepetitions}`	Total
`RepetitionInfo`	Inject repetition details

⚠ COMMON MISTAKES

- Expecting `@BeforeEach` to run before every dynamic test
- Dynamic tests whose names collide, making reports confusing
- Repeating slow tests 1000 times in the normal suite

✓ TIPS & BEST PRACTICES

- Use dynamic tests for data discovered at run time; parameterized tests for fixed data
- Tag repeated flake hunts so they don't run in CI
- Give dynamic tests names that identify the input

QUICK SUMMARY

- `@TestFactory` — tests at run time
- `dynamicContainer` — grouped
- `@RepeatedTest` — run N times
- `failureThreshold` — stop early

The Extension Model

`@ExtendWith` · `@RegisterExtension` · lifecycle callbacks · `ParameterResolver` · `ExtensionContext.Store` · writing a small extension

PLUG BEHAVIOR INTO THE LIFECYCLE

Everything JUnit 4 did with runners and rules, Jupiter does with **extensions**: classes that implement callback interfaces (`BeforeEachCallback`, `AfterTestExecutionCallback`…) or **resolve parameters**. Register them declaratively with **`@ExtendWith`**, or programmatically with **`@RegisterExtension`** on a field when they need configuration. Mockito, Spring and Testcontainers all integrate this way.

NOTES

- Several extensions can be combined; they run in registration order
- `ExtensionContext.Store` keeps state per test, class or run
- Meta-annotations bundle extensions: `@ExtendWith(...)` `@interface IntegrationTest`
- Global auto-registration via `ServiceLoader` (opt-in)

IN SPRING BOOT

- `@SpringBootTest` is meta-annotated with `@ExtendWith(SpringExtension.class)`
- SpringExtension resolves `@Autowired` constructor and method parameters
- Order matters: put `MockitoExtension` after Spring if both are present

USING AND WRITING EXTENSIONS

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest { }

class TimingExtension implements BeforeTestExecutionCallback, AfterTestExecutionCallback {
    private static final Namespace NS = Namespace.create(TimingExtension.class);

    public void beforeTestExecution(ExtensionContext ctx) {
        ctx.getStore(NS).put("start", System.nanoTime());
    }
    public void afterTestExecution(ExtensionContext ctx) {
        long start = ctx.getStore(NS).remove("start", long.class);
        ctx.publishReportEntry("ms", String.valueOf((System.nanoTime() - start) / 1_000_000));
    }
}

class ClockResolver implements ParameterResolver {
    public boolean supportsParameter(ParameterContext p, ExtensionContext e) {
        return p.getParameter().getType() == Clock.class;
    }
    public Object resolveParameter(ParameterContext p, ExtensionContext e) {
        return Clock.fixed(Instant.parse("2026-11-04T12:00:00Z"), ZoneOffset.UTC);
    }
}

@ExtendWith({TimingExtension.class, ClockResolver.class})
class InvoiceTest {
    @RegisterExtension
    static WireMockExtension wm = WireMockExtension.newInstance().build();

    @Test void dueInThirtyDays(Clock clock) { /* clock injected */ }
}
```

CODE EXPLAINED

- The `Store` keeps the start time between the two callbacks for this test
- `publishReportEntry` adds data to test reports
- A `ParameterResolver` lets tests and lifecycle methods ask for a `Clock`
- `@RegisterExtension` on a static field configures an extension instance in code

JUNIT 4 / TESTNG EQUIVALENT

- JUnit 4 `@RunWith(MockitoJUnitRunner.class)` (one runner per class) and `@Rule` / `@ClassRule` fields
- TestNG `@Listeners`, `ITestListener`, `IHookable`

PREFER / AVOID

- abstract base test classes for shared setup → extensions or meta-annotations
- static helper state across tests → `ExtensionContext.Store`
- manual `Clock` setup in every test → a `ParameterResolver`

CALLBACK INTERFACES

INTERFACE	RUNS
`BeforeAllCallback / AfterAllCallback`	Once per class
`BeforeEachCallback / AfterEachCallback`	Around @BeforeEach / @AfterEach
`BeforeTestExecutionCallback`	Right before the test body
`AfterTestExecutionCallback`	Right after the test body

OTHER EXTENSION POINTS

INTERFACE	DOES
`ParameterResolver`	Supply parameters
`ExecutionCondition`	Enable / disable tests
`TestExecutionExceptionHandler`	Handle exceptions
`TestWatcher`	React to results
`InvocationInterceptor`	Wrap invocations
`TestInstancePostProcessor`	Inject into test instances

⚠ COMMON MISTAKES

- Storing state in static fields of an extension, which breaks with parallel tests
- Assuming extension order when registering with several annotations
- Recreating Mockito / Spring integration by hand instead of using their extensions

✓ TIPS & BEST PRACTICES

- Bundle common extensions into a meta-annotation like `@IntegrationTest`
- Use the `Store` with a unique `Namespace`
- Check JUnit Pioneer and library docs before writing your own

QUICK SUMMARY

- `@ExtendWith` — declarative
- `@RegisterExtension` — configured instance
- callbacks + `ParameterResolver`
- `ExtensionContext.Store` — per-test state

Built-in Parameters & Extensions

THINGS JUNIT GIVES YOU FOR FREE

JUnit resolves some parameters on its own: a fresh temporary directory with **`@TempDir`**, **`TestInfo`** with the current test's name and tags, and **`TestReporter`** to publish key-value data. JUnit 6.1 also adds built-in extensions for **default locale and time zone** and for setting and restoring **system properties**, which used to need JUnit Pioneer.

NOTES

- `@TempDir` works on fields and parameters, `Path` or `File`
- Cleanup modes: `ALWAYS` (default), `ON_SUCCESS`, `NEVER`
- Locale and time-zone extensions restore the original afterwards
- Check the 6.1 user guide for the exact system-property annotations

IN SPRING BOOT

- Spring `@DynamicPropertySource` can point properties at a `@TempDir` path for file-based beans
- Boot's `@SpringBootTest(properties = …)` sets properties per test class without touching system properties

BUILT-INS

```
@Test
void writesReport(@TempDir Path dir) throws IOException {
    Path out = dir.resolve("report.csv");
    Reports.export(out);
    assertThat(Files.readAllLines(out)).first().isEqualTo("sku,qty");
}

@TempDir(cleanup = CleanupMode.ON_SUCCESS)    // keep files when a test fails
Path workDir;

@Test
@Tag("fast")
void info(TestInfo info, TestReporter reporter) {
    assertTrue(info.getTags().contains("fast"));
    reporter.publishEntry("user", "ann");      // appears in reports
}

@Test
@DefaultLocale("de-DE")                        // JUnit 6.1
@DefaultTimeZone("Europe/Berlin")
void formatsGermanDates() {
    assertEquals("04.11.2026", Dates.format(LocalDate.of(2026, 11, 4)));
}
```

CODE EXPLAINED

- → Each test gets its own `@TempDir`, deleted afterwards (unless the cleanup mode keeps it)
- → `ON_SUCCESS` leaves the files behind for debugging failed tests
- → `TestInfo` exposes display name, tags, class and method
- → Locale and time zone are restored after the test, so other tests aren't affected

JUNIT 4 / TESTNG EQUIVALENT

- ↺ **JUnit 4** `TemporaryFolder` rule, `TestName` rule; no locale helpers
- ↺ **TestNG** No built-in temp dir; `ITestContext` / `ITestResult` for info; set locale manually

PREFER / AVOID

- ⇄ `Files.createTempDirectory` + manual cleanup → `@TempDir`
- ⇄ `Locale.setDefault` without restoring → `@DefaultLocale`
- ⇄ `System.setProperty` in tests → the 6.1 system-property extension or Spring test properties

BUILT-IN PARAMETERS

PARAMETER	GIVES
`@TempDir Path`	Temporary directory
`TestInfo`	Name, tags, class, method
`TestReporter`	Publish report entries
`RepetitionInfo`	Repetition number (sheet 13)

@TEMPDIR OPTIONS

OPTION	EFFECT
`cleanup = ALWAYS`	Delete after test (default)
`cleanup = ON_SUCCESS`	Keep on failure
`cleanup = NEVER`	Keep always
`factory = …`	Custom location / file system
`junit.jupiter.tempdir.cleanup.mode.default`	Project default

⚠ COMMON MISTAKES

- Writing test files to the project folder or `/tmp` with fixed names
- Changing the default locale or time zone without restoring it
- Relying on the machine's locale for date or number formatting in assertions

✓ TIPS & BEST PRACTICES

- Use `ON_SUCCESS` cleanup while debugging file-producing code
- Pin locale and time zone in formatting tests
- Publish IDs and URLs with `TestReporter` to make CI failures easier to trace

QUICK SUMMARY

- `@TempDir` — temp folder
- `TestInfo` / `TestReporter` — context and report data
- `@DefaultLocale` / `@DefaultTimeZone` (6.1)
- built-ins restore state afterwards

Sheet 16

Parallel Execution

junit.jupiter.execution.parallel.enabled · mode.default · @Execution · @ResourceLock · @Isolated · thread-safety

RUN TESTS ON SEVERAL THREADS

JUnit can run tests in **parallel** inside one JVM. Turn it on in `junit-platform.properties`, choose whether methods and classes run concurrently by default, then mark exceptions: **`@Execution(SAME_THREAD)`** for tests that must stay serial, **`@ResourceLock`** for tests sharing a resource, and **`@Isolated`** for a class that must run alone.

NOTES

- Off by default; tests must be thread-safe first
- Strategy `dynamic` sizes the pool from CPU cores
- Build-tool forking (Surefire `forkCount`, Gradle `maxParallelForks`) is a separate, process-level option
- JUnit 6.1 adds a new parallel executor implementation

IN SPRING BOOT

- Spring's cached context is shared across threads; beans with mutable state break under parallel tests
- `@DirtiesContext` and `@MockitoBean` changes don't play well with concurrent classes
- Many teams keep `@SpringBootTest` classes `SAME_THREAD` and parallelize plain unit tests

CONFIG AND ANNOTATIONS

```
# src/test/resources/junit-platform.properties
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default = concurrent
junit.jupiter.execution.parallel.mode.classes.default = concurrent
junit.jupiter.execution.parallel.config.strategy = dynamic
junit.jupiter.execution.parallel.config.dynamic.factor = 1

# In code:
# @Execution(ExecutionMode.SAME_THREAD)       — this class / method runs serially
# @ResourceLock(value = "db", mode = READ_WRITE)
# @ResourceLock(Resources.SYSTEM_PROPERTIES)  — built-in lock names
# @Isolated                                   — nothing else runs at the same time

# Process-level parallelism (separate JVMs):
# Maven Surefire: <forkCount>2</forkCount> <reuseForks>true</reuseForks>
# Gradle:        tasks.test { maxParallelForks = 2 }
```

CODE EXPLAINED

- → `mode.default` controls methods within a class; `mode.classes.default` controls classes
- → `dynamic` with factor 1 uses one thread per core
- → `@ResourceLock` serializes only the tests that share the named resource
- → Forking runs separate JVMs, which isolates static state but costs memory

JUNIT 4 / TESTNG EQUIVALENT

- JUnit 4 — No in-JVM parallel support (Surefire `parallel` option with JUnit 4 provider)
- TestNG — `parallel="methods"` / `thread-count` in `testng.xml`

PREFER / AVOID

- ⇄ global `SAME_THREAD` because of one bad test → `@ResourceLock` or `@Isolated` on that test
- ⇄ static mutable fields → instance state per test
- ⇄ turning on parallel mode without checking → run with `--count` /repeats first to flush races

SETTINGS

KEY	VALUES
`parallel.enabled`	true / false
`mode.default`	same_thread / concurrent
`mode.classes.default`	same_thread / concurrent
`config.strategy`	dynamic / fixed / custom
`config.fixed.parallelism`	Thread count (fixed)

BUILT-IN LOCK NAMES (RESOURCES)

CONSTANT	PROTECTS
`SYSTEM_PROPERTIES`	System.getProperties()
`SYSTEM_OUT / SYSTEM_ERR`	Console streams
`LOCALE`	Default locale
`TIME_ZONE`	Default time zone
`GLOBAL`	Everything (like @Isolated)

⚠ COMMON MISTAKES

- Enabling parallel mode with tests that share static state or files
- Parallel tests writing to `System.out` and asserting on captured output
- Expecting `@BeforeAll` static setup to be thread-safe automatically

✓ TIPS & BEST PRACTICES

- Start with classes concurrent and methods same-thread, then widen
- Lock shared resources explicitly with `@ResourceLock`
- Measure; parallelism helps CPU-bound suites more than I/O-bound ones

QUICK SUMMARY

- `parallel.enabled = true`
- `mode.default` / `mode.classes.default`
- `@ResourceLock` / `@Isolated` — exceptions
- forking — separate JVMs

Mockito Basics

`MockitoExtension` · `@Mock` · `@InjectMocks` · `when(…).thenReturn` · `verify` · `any()` / `eq()` · `times()` / `never()`

FAKE COLLABORATORS, CHECK INTERACTIONS

Mockito creates **mocks**: objects that implement a class or interface, return defaults (null, 0, empty) until **stubbed**, and record every call. With **`MockitoExtension`**, `@Mock` fields are created for each test and **`@InjectMocks`** builds the class under test with them. Stub with `when(...).thenReturn(...)` and check calls with `verify`.

NOTES

- Add `mockito-junit-jupiter` (Spring Boot includes it)
- Mockito 5 uses the inline mock maker: final classes and methods can be mocked
- Strict stubs: unused stubbing fails the test (`UnnecessaryStubbingException`)
- If you use one matcher, every argument needs a matcher

IN SPRING BOOT

- `@MockitoBean` replaces a bean in the Spring context with a Mockito mock; `@Mock` only creates a plain mock
- Use plain `@Mock` + `@InjectMocks` for unit tests; it's much faster than starting Spring

A SERVICE TEST WITH MOCKS

```java
import static org.mockito.Mockito.*;
import static org.mockito.ArgumentMatchers.*;

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository repo;
    @Mock Mailer mailer;
    @InjectMocks OrderService service;      // new OrderService(repo, mailer)

    @Test
    void placesOrderAndSendsMail() {
        when(repo.save(any(Order.class))).thenAnswer(inv -> inv.getArgument(0));

        Order o = service.place(new Cart("ann@example.com"));

        assertEquals(OrderStatus.NEW, o.status());
        verify(repo).save(o);
        verify(mailer).send(eq("ann@example.com"), contains("Order"));
        verifyNoMoreInteractions(mailer);
    }

    @Test
    void unknownOrder() {
        when(repo.findById(99L)).thenReturn(Optional.empty());
        assertThrows(OrderNotFoundException.class, () -> service.get(99L));
        verify(mailer, never()).send(any(), any());
    }
}
```

CODE EXPLAINED

- → `@InjectMocks` uses constructor injection when it can
- → `thenAnswer` returns the saved order itself, like a real repository would
- → `verify(repo).save(o)` checks the call happened once with that argument
- → `never()` asserts no email was sent

JUNIT 4 / TESTNG EQUIVALENT

- ↺ JUnit 4 `@RunWith(MockitoJUnitRunner.class)` or `MockitoJUnit.rule()`
- ↺ TestNG `MockitoAnnotations.openMocks(this)` in `@BeforeMethod`, or `MockitoTestNGListener`

PREFER / AVOID

- ⇄ `Mockito.mock()` in every test → `@Mock` fields + extension
- ⇄ mocking value objects and DTOs → real instances
- ⇄ verifying every call → verify only the interactions that matter

STUBBING

CALL	EFFECT
`when(m.x()).thenReturn(v)`	Return v
`…thenReturn(a, b)`	a, then b
`…thenThrow(new E())`	Throw
`…thenAnswer(inv -> …)`	Compute
`doReturn(v).when(m).x()`	Stub spies / void-safe

VERIFYING

CALL	CHECKS
`verify(m).x()`	Called once
`verify(m, times(2)).x()`	Twice
`verify(m, never()).x()`	Never
`verify(m, atLeastOnce()).x()`	≥ 1
`verifyNoMoreInteractions(m)`	Nothing else
`any()` / `eq(v)` / `argThat(p)`	Argument matchers

⚠ COMMON MISTAKES

- Mixing raw values and matchers: `verify(m).send("a", any())` throws; use `eq("a")`
- Stubbing things the test never uses (strict stubs fail it)
- Mocking the class under test

✓ TIPS & BEST PRACTICES

- Mock at boundaries (repositories, HTTP clients, clocks), not your domain objects
- Prefer asserting on results; verify only important side effects
- Add Mockito as a Java agent in the build to avoid JDK 21+ dynamic-agent warnings

QUICK SUMMARY

- `@Mock` + `@InjectMocks` + `MockitoExtension`
- `when(…).thenReturn(…)` — stub
- `verify(m, times(n))` — check calls
- matchers — all or none

Mockito Advanced

ArgumentCaptor · @Spy · doThrow / doAnswer · BDDMockito given / then · strict stubs · mockStatic

CAPTURING, SPYING AND STATIC MOCKS

An **ArgumentCaptor** grabs the argument a mock received so you can assert on it in detail. A **@Spy** wraps a real object: real methods run unless stubbed. The **do…().when()** family stubs `void` methods and spies safely. **BDDMockito** reads as given/when/then, and **mockStatic** fakes a static method inside a try-with-resources block.

NOTES

- Captors: `@Captor ArgumentCaptor<Order> captor;`
- Stub spies with `doReturn`, not `when`, to avoid calling the real method
- `mockStatic` / `mockConstruction` need the inline mock maker (default in Mockito 5)
- Strictness: `@MockitoSettings(strictness = Strictness.LENIENT)` loosens checks

IN SPRING BOOT

- `@MockitoBean` mocks are reset after each test automatically (`MockReset.AFTER`)
- Strict-stub checks apply to `@MockitoBean` mocks too
- `@MockitoSpyBean` wraps a real bean, like `@Spy` inside the context

CAPTORS, SPIES, BDD, STATIC

```java
import static org.mockito.BDDMockito.*;

@ExtendWith(MockitoExtension.class)
class CheckoutTest {
    @Mock PaymentGateway gateway;
    @Spy PriceCalculator calc = new PriceCalculator();
    @Captor ArgumentCaptor<Charge> chargeCaptor;
    @InjectMocks Checkout checkout;

    @Test
    void chargesTheRightAmount() {
        given(gateway.charge(any())).willReturn(new Receipt("r1"));   // BDD stub

        checkout.pay(cart);

        then(gateway).should().charge(chargeCaptor.capture());        // BDD verify
        Charge c = chargeCaptor.getValue();
        assertThat(c.amountCents()).isEqualTo(1999);
        assertThat(c.currency()).isEqualTo("USD");
        verify(calc).total(cart);                                     // real method ran
    }

    @Test
    void gatewayDown() {
        doThrow(new GatewayException()).when(gateway).refund(anyString());
        assertThrows(RefundFailed.class, () -> checkout.refund("r1"));
    }

    @Test
    void fixedNow() {
        try (MockedStatic<Instant> mocked = mockStatic(Instant.class, CALLS_REAL_METHODS)) {
            mocked.when(Instant::now).thenReturn(Instant.parse("2026-11-04T00:00:00Z"));
            assertThat(checkout.receiptDate()).isEqualTo("2026-11-04");
        }                                                             // restored here
    }
}
```

CODE EXPLAINED

- → The captor holds the `Charge` passed to the gateway so its fields can be checked
- → `@Spy` lets the real `total` run while still recording the call
- → `doThrow(...).when(mock).voidMethod()` is how to stub `void` methods
- → `mockStatic` is scoped to the try block; injecting a `Clock` is usually cleaner

JUNIT 4 / TESTNG EQUIVALENT

- ↺ **JUnit 4** Same Mockito API; runner or rule instead of the extension
- ↺ **TestNG** Same Mockito API; `MockitoTestNGListener` for annotations

PREFER / AVOID

- ⇄ `mockStatic` for time → inject a `Clock`
- ⇄ `when(spy.x())` → `doReturn(v).when(spy).x()`
- ⇄ captors for simple equality → `verify(m).x(eq(expected))`

DO… FAMILY

CALL	USE FOR
`doReturn(v).when(m).x()`	Spies, avoiding real call
`doThrow(e).when(m).x()`	Void methods that throw
`doNothing().when(m).x()`	Void on spies
`doAnswer(inv -> …).when(m).x()`	Side effects, callbacks
`doCallRealMethod().when(m).x()`	Real method on a mock

CLASSIC VS BDD

CLASSIC	BDDMOCKITO
`when(m.x()).thenReturn(v)`	given(m.x()).willReturn(v)
`doThrow(e).when(m).x()`	willThrow(e).given(m).x()
`verify(m).x()`	then(m).should().x()
`verify(m, never()).x()`	then(m).should(never()).x()

⚠ COMMON MISTAKES

- `when(spy.method())` calling the real method during stubbing
- Forgetting to close `MockedStatic`, leaking the mock into other tests
- Captors used where a simple matcher would read better

✓ TIPS & BEST PRACTICES

- Use BDD style if your team writes given/when/then test names
- Wrap static APIs (time, UUIDs) behind injectable interfaces
- Use `lenient()` on a single stub instead of loosening the whole class

QUICK SUMMARY

- `ArgumentCaptor` — inspect arguments
- `@Spy` — real object + recording
- `do…().when()` — voids and spies
- `mockStatic` — scoped static mocks

Sheet 19

Spring Boot Test Basics

`@SpringBootTest` · `webEnvironment` · `@MockitoBean` · `@TestConfiguration` · `@ActiveProfiles` · context caching

TEST WITH THE WHOLE APPLICATION CONTEXT

`@SpringBootTest` starts your Spring Boot application context for a test class, so beans are real and wired together. With `webEnvironment = RANDOM_PORT` it starts a real server too. **`@MockitoBean`** swaps one bean for a mock, **`@TestConfiguration`** adds test-only beans, and **`@ActiveProfiles`** picks configuration. Contexts are **cached** between classes with identical setup, so keep setups consistent. The Spring set covers the app side.

NOTES

- `@MockBean` was deprecated in Boot 3.4 and removed in Boot 4; use `@MockitoBean`
- Each distinct context configuration costs a full start-up
- Use slices (sheet 20) when you only need one layer
- Test properties: `@SpringBootTest(properties = "feature.x=true")`

FULL-CONTEXT TESTS

```
@SpringBootTest
@ActiveProfiles("test")
class CheckoutFlowTest {
    @Autowired CheckoutService checkout;
    @MockitoBean PaymentGateway gateway;          // replaces the real bean

    @Test
    void chargesAndStoresOrder() {
        given(gateway.charge(any())).willReturn(new Receipt("r1"));
        Order o = checkout.pay(sampleCart());
        assertThat(o.status()).isEqualTo(OrderStatus.PAID);
    }
}

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class HealthEndpointTest {
    @Autowired WebTestClient client;              // or TestRestTemplate

    @Test
    void healthIsUp() {
        client.get().uri("/actuator/health").exchange()
              .expectStatus().isOk()
              .expectBody().jsonPath("$.status").isEqualTo("UP");
    }
}

@TestConfiguration
class FixedClockConfig {
    @Bean Clock clock() { return Clock.fixed(Instant.parse("2026-11-04T00:00:00Z"), ZoneOffset.UTC); }
}
```

CODE EXPLAINED

- → `@MockitoBean` mocks are reset after each test
- → `RANDOM_PORT` starts a real embedded server on a free port
- → `@TestConfiguration` beans are added only when imported or found as nested classes
- → Every class with the same annotations reuses one cached context

JUNIT 4 / TESTNG EQUIVALENT

- ↺ **JUnit 4** `@RunWith(SpringRunner.class)` + `@SpringBootTest` ; `@MockBean`
- ↺ **TestNG** `AbstractTestNGSpringContextTests` base class + `@SpringBootTest`

PREFER / AVOID

- ⇄ `@MockBean` → `@MockitoBean`
- ⇄ `@SpringBootTest` for a controller test → `@WebMvcTest` slice
- ⇄ `@DirtiesContext` everywhere → reset state explicitly, keep the cache

WEBENVIRONMENT

VALUE	STARTS
`MOCK (default)`	Mock servlet environment, no server
`RANDOM_PORT`	Real server, random port
`DEFINED_PORT`	Real server, configured port
`NONE`	No web environment

COMMON ANNOTATIONS

ANNOTATION	EFFECT
`@MockitoBean`	Replace bean with a mock
`@MockitoSpyBean`	Wrap a real bean
`@TestConfiguration`	Extra test beans
`@ActiveProfiles("test")`	Profile
`@DynamicPropertySource`	Properties computed at run time
`@DirtiesContext`	Drop cached context (slow)

⚠ COMMON MISTAKES

- Slightly different `@MockitoBean` sets per class, each forcing a new context
- Using `@SpringBootTest` for every test and making the suite take minutes
- Tests that change shared beans and break later classes using the cached context

✓ TIPS & BEST PRACTICES

- Keep a few standard test base annotations so contexts are reused
- Use slices or plain unit tests for most cases
- Watch the log for "Started … in N seconds" to count context starts

QUICK SUMMARY

- `@SpringBootTest` — whole context
- `@MockitoBean` — replace a bean
- `RANDOM_PORT` — real server
- context caching — keep setups identical

Spring Test Slices

START ONLY THE LAYER YOU'RE TESTING

A **test slice** starts a small part of the context: **`@WebMvcTest`** loads controllers and MVC config with **`MockMvc`** / **`MockMvcTester`**, **`@DataJpaTest`** loads JPA repositories with a test database, **`@JsonTest`** loads JSON mapping, **`@RestClientTest`** loads HTTP clients with a mock server. Slices start much faster than `@SpringBootTest` and keep tests focused.

NOTES

- Beans outside the slice must be mocked with `@MockitoBean`
- `@DataJpaTest` runs each test in a rolled-back transaction
- `MockMvcTester` (Spring 6.2+) gives AssertJ-style checks
- Boot 4 moved slice annotations into per-technology modules; check imports

A CONTROLLER SLICE AND A JPA SLICE

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvcTester mvc;
    @MockitoBean OrderService service;

    @Test
    void returnsOrder() {
        given(service.get(1L)).willReturn(new OrderDto(1L, "PAID"));
        assertThat(mvc.get().uri("/orders/1"))
            .hasStatusOk()
            .bodyJson().extractingPath("$.status").isEqualTo("PAID");
    }

    @Test
    void validatesInput() {
        assertThat(mvc.post().uri("/orders")
                      .contentType(MediaType.APPLICATION_JSON).content("{}"))
            .hasStatus(HttpStatus.BAD_REQUEST);
    }
}

@DataJpaTest
class OrderRepositoryTest {
    @Autowired OrderRepository repo;
    @Autowired TestEntityManager em;

    @Test
    void findsPaidOrders() {
        em.persist(new Order(OrderStatus.PAID));
        em.persist(new Order(OrderStatus.NEW));
        assertThat(repo.findByStatus(OrderStatus.PAID)).hasSize(1);
    }
}
```

CODE EXPLAINED

- → `@WebMvcTest(OrderController.class)` loads just that controller plus MVC infrastructure
- → The service is mocked because services aren't part of the web slice
- → `bodyJson().extractingPath(...)` reads a JSON path from the response
- → `TestEntityManager` sets up data; each test rolls back

JUNIT 4 / TESTNG EQUIVALENT

- ↺ **JUnit 4** Same slices with `@RunWith(SpringRunner.class)`
- ↺ **TestNG** Same slices with `AbstractTestNGSpringContextTests`

PREFER / AVOID

- ⇄ `@SpringBootTest` + `MockMvc` for controller tests → `@WebMvcTest`
- ⇄ in-memory H2 for `@DataJpaTest` when prod is Postgres → Testcontainers (sheet 21)
- ⇄ classic `MockMvc.perform(...).andExpect(...)` → `MockMvcTester` with AssertJ

SLICES

ANNOTATION	LOADS
`@WebMvcTest`	Controllers, filters, MVC config
`@WebFluxTest`	WebFlux controllers + WebTestClient
`@DataJpaTest`	JPA repositories, test DB
`@JdbcTest` / `@DataJdbcTest`	JDBC / Spring Data JDBC
`@JsonTest`	Jackson / JSON mapping
`@RestClientTest`	RestClient + MockRestServiceServer
`@DataMongoTest` / `@DataRedisTest`	NoSQL repositories

MOCKMVCTESTER CHECKS

CHECK	EXAMPLE
`Status`	.hasStatusOk() / .hasStatus(400)
`JSON path`	.bodyJson().extractingPath("$.id")
`Header`	.hasHeader("Location", …)
`View / model`	.hasViewName("orders")
`Redirect`	.hasRedirectedUrl("/login")

⚠ COMMON MISTAKES

- Expecting services to be injected in `@WebMvcTest` without mocking them
- `@DataJpaTest` against H2 hiding Postgres-specific SQL bugs
- Pulling the whole app into a slice with `@Import` of large configs

✓ TIPS & BEST PRACTICES

- Write most web tests as `@WebMvcTest` and a few as full `@SpringBootTest`
- Use `@AutoConfigureTestDatabase(replace = NONE)` with Testcontainers in `@DataJpaTest`
- Keep controllers thin so slice tests stay simple

QUICK SUMMARY

- slice — one layer of the context
- `@WebMvcTest` + `MockMvcTester`
- `@DataJpaTest` — repositories, rolled back
- mock everything outside the slice

Testcontainers

REAL DATABASES AND SERVICES IN DOCKER

Testcontainers starts real services (PostgreSQL, Kafka, Redis, any image) in Docker for your tests and throws them away afterwards. Tests run against the same database engine as production instead of an in-memory stand-in. In Spring Boot, **`@ServiceConnection`** wires the container's URL and credentials into your beans automatically.

NOTES

- Needs Docker (or a compatible runtime) on the machine and CI runner
- Static `@Container` fields start once per class; instance fields once per test
- Testcontainers 2 reorganized module packages; check imports when upgrading
- Reuse containers across runs locally with `withReuse(true)` + `testcontainers.reuse.enable=true`

POSTGRESQL IN TESTS

```
@SpringBootTest
@Testcontainers
class OrderRepositoryIT {

    @Container
    @ServiceConnection                                  // Boot sets spring.datasource.*
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");

    @Autowired OrderRepository repo;

    @Test
    void savesAndLoads() {
        Order saved = repo.save(new Order(OrderStatus.NEW));
        assertThat(repo.findById(saved.getId())).isPresent();
    }
}

// Without @ServiceConnection (non-standard properties):
@DynamicPropertySource
static void props(DynamicPropertyRegistry r) {
    r.add("app.search.url", () -> "http://" + es.getHost() + ":" + es.getMappedPort(9200));
}

// Generic container
@Container
static GenericContainer<?> redis = new GenericContainer<>("redis:7-alpine")
        .withExposedPorts(6379);
```

CODE EXPLAINED

- → `@Testcontainers` starts and stops `@Container` fields around the class
- → `@ServiceConnection` knows how to configure Spring for common images
- → `@DynamicPropertySource` passes container ports to custom properties
- → Pin image tags (`postgres:17-alpine`) so tests don't change under you

JUNIT 4 / TESTNG EQUIVALENT

- ↺ **JUnit 4** `@ClassRule public static PostgreSQLContainer…` with Testcontainers' JUnit 4 support
- ↺ **TestNG** Start containers in `@BeforeClass` and stop in `@AfterClass` manually

PREFER / AVOID

- ⇄ H2 pretending to be Postgres → `PostgreSQLContainer`
- ⇄ one container per test method → static containers per class or a shared singleton
- ⇄ `latest` image tags → pinned versions

COMMON MODULES

MODULE	CONTAINER
`postgresql`	PostgreSQLContainer
`mysql`	MySQLContainer
`kafka`	KafkaContainer
`mongodb`	MongoDBContainer
`localstack`	LocalStackContainer (AWS)
`(core)`	GenericContainer for any image

KEEPING IT FAST

TECHNIQUE	HOW
`Static containers`	Once per class
`Singleton pattern`	Start once in a base class, never stop
`Reuse`	withReuse(true) locally
`Alpine / slim images`	Smaller pulls
`Context caching`	Same Spring setup across ITs

⚠ COMMON MISTAKES

- Running Testcontainers tests where Docker isn't available and getting confusing errors
- Non-static `@Container` fields starting a fresh database for every test
- Mixing up container-internal ports and mapped host ports

✓ TIPS & BEST PRACTICES

- Name integration tests `*IT` and run them with Failsafe / a separate Gradle task
- Use `@EnabledIf` with Docker availability (sheet 10) for laptops without Docker
- Prefer `@ServiceConnection` over hand-written properties

QUICK SUMMARY

- Testcontainers — real services in Docker
- `@Container` static — once per class
- `@ServiceConnection` — auto-wire Spring
- `@DynamicPropertySource` — custom properties

Migrating from JUnit 4 & 5

@Before → @BeforeEach · @Rule → extensions · expected= → assertThrows · @RunWith → @ExtendWith · Vintage engine · 5 → 6 changes

MOVING TO JUPITER AND JUNIT 6

JUnit 4 tests can run unchanged on the **Vintage** engine while you migrate, but Vintage is deprecated in JUnit 6, so plan to move them. Most changes are mechanical: new packages, renamed annotations, message-last assertions, and runners and rules replaced by extensions. **OpenRewrite** recipes automate much of it. Going from **JUnit 5 to 6** is small for most suites: Java 17, unified versions, and removed deprecated APIs.

NOTES

- Run old and new tests side by side with Vintage during the move
- OpenRewrite: `org.openrewrite.java.testing.junit5.JUnit4to5Migration`
- IntelliJ has a "Migrate to JUnit 5" inspection
- Hamcrest still works; many teams switch to AssertJ at the same time

JUNIT 4 → JUPITER

```
// JUnit 4                                  // JUnit Jupiter (5 / 6)
import org.junit.*;                         import org.junit.jupiter.api.*;

public class CartTest {                     class CartTest {
  @Rule public TemporaryFolder tmp          @TempDir Path tmp;
      = new TemporaryFolder();
  @BeforeClass public static void a()       @BeforeAll static void a()
  @Before public void setUp()               @BeforeEach void setUp()
  @After public void tearDown()             @AfterEach void tearDown()
  @Ignore("why")                            @Disabled("why")
  @Category(Slow.class)                     @Tag("slow")

  @Test(expected = IAE.class)               @Test void rejects() {
  public void rejects() {                     assertThrows(IAE.class, () -> cart.add("A1", 0));
    cart.add("A1", 0);                      }
  }

  @Test(timeout = 1000)                     @Test @Timeout(1)
  assertEquals("msg", 2, n);                assertEquals(2, n, "msg");
}                                           }

@RunWith(MockitoJUnitRunner.class)     →    @ExtendWith(MockitoExtension.class)
@RunWith(SpringRunner.class)           →    (implied by @SpringBootTest)
```

CODE EXPLAINED

- → Annotations live in `org.junit.jupiter.api`; assertions in `Assertions`
- → The assertion message moves from the first argument to the last
- → Rules become extensions or built-ins (`@TempDir` , `@Timeout`)
- → Classes and methods no longer need `public`

JUNIT 4 / TESTNG EQUIVALENT

- ↺ **JUnit 4** This sheet is the JUnit 4 map
- ↺ **TestNG** TestNG → Jupiter: `@BeforeMethod` → `@BeforeEach` , `dataProvider` → `@MethodSource` , `groups` → `@Tag` , `expectedExceptions` → `assertThrows`

PREFER / AVOID

- ⇄ hand-editing hundreds of files → OpenRewrite / IDE migration
- ⇄ keeping Vintage forever → migrate and drop it before it's removed
- ⇄ migrating and refactoring tests at once → migrate first, then improve

ANNOTATION MAP

JUNIT 4	JUPITER
`@Before / @After`	@BeforeEach / @AfterEach
`@BeforeClass / @AfterClass`	@BeforeAll / @AfterAll
`@Ignore`	@Disabled
`@Category`	@Tag
`@RunWith`	@ExtendWith
`@Rule / @ClassRule`	Extensions / @RegisterExtension
`@Test(expected=)`	assertThrows
`@Test(timeout=)`	@Timeout

JUNIT 5 → 6

CHANGE	ACTION
`Java 17 baseline`	Upgrade JDK first
`One version for all modules`	Use junit-bom 6.x
`Vintage deprecated`	Finish JUnit 4 migration
`Deprecated 5.x APIs removed`	Fix deprecation warnings on 5.14 first
`JSpecify nullness`	Kotlin / null-checkers may flag code

⚠ COMMON MISTAKES

- Mixing JUnit 4 and Jupiter imports in one class
- Leaving the message as the first argument after migration (it becomes the "expected" value)
- Forgetting the Vintage engine during migration, so JUnit 4 tests silently stop running

✓ TIPS & BEST PRACTICES

- Upgrade to the last 5.x release and clear deprecation warnings before moving to 6
- Track test counts before and after migration
- Remove `junit:junit` once nothing imports it

QUICK SUMMARY

- `@Before` → `@BeforeEach` , `@Ignore` → `@Disabled`
- rules / runners → extensions
- message moves last
- JUnit 6 — Java 17, Vintage deprecated

Reports, Coverage & CI

Surefire XML reports · Open Test Reporting · JaCoCo · coverage thresholds · GitHub Actions · test result publishing

RESULTS YOUR CI CAN READ

Maven Surefire and Gradle write **JUnit XML** reports that every CI system understands. JUnit's **Open Test Reporting** format adds richer events. **JaCoCo** measures code coverage and can fail the build below a threshold. In CI, run the build's test task, publish the reports, and keep the artifacts when tests fail.

NOTES

- Surefire XML: `target/surefire-reports/TEST-*.xml`
- Gradle XML: `build/test-results/test/` ; HTML: `build/reports/tests/test/`
- Enable Open Test Reporting: `junit.platform.reporting.open.xml.enabled=true`
- The Maven, Gradle and GitHub sets cover each tool

JACOCO AND A WORKFLOW

```
// build.gradle.kts
plugins { java; jacoco }
tasks.test {
    useJUnitPlatform()
    finalizedBy(tasks.jacocoTestReport)
}
tasks.jacocoTestReport { reports { xml.required = true; html.required = true } }
tasks.jacocoTestCoverageVerification {
    violationRules { rule { limit { minimum = "0.80".toBigDecimal() } } }
}
tasks.check { dependsOn(tasks.jacocoTestCoverageVerification) }

# .github/workflows/test.yml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-java@v5
        with: { distribution: temurin, java-version: 21, cache: gradle }
      - run: ./gradlew check
      - uses: actions/upload-artifact@v7
        if: always()
        with: { name: test-reports, path: build/reports/ }
```

CODE EXPLAINED

- → `finalizedBy` builds the coverage report after every test run
- → `jacocoTestCoverageVerification` fails `check` below 80% line coverage
- → `cache: gradle` speeds up dependency downloads in CI
- → Reports upload even when tests fail (`if: always()`)

JUNIT 4 / TESTNG EQUIVALENT

- ↺ **JUnit 4** Same Surefire / Gradle XML reports and JaCoCo setup
- ↺ **TestNG** TestNG writes its own `testng-results.xml` plus JUnit-style XML

PREFER / AVOID

- ⇄ reading raw logs for failures → publish JUnit XML to the CI test tab
- ⇄ coverage only on a laptop → JaCoCo in `check`
- ⇄ artifacts only on success → `if: always()`

REPORT LOCATIONS

TOOL	PATH
Surefire	target/surefire-reports/
Failsafe	target/failsafe-reports/
Gradle XML	build/test-results/test/
Gradle HTML	build/reports/tests/test/
JaCoCo (Maven)	target/site/jacoco/
JaCoCo (Gradle)	build/reports/jacoco/

JACOCO (MAVEN)

GOAL	DOES
prepare-agent	Attach the agent for tests
report	HTML / XML report
check	Fail below rules
report-aggregate	Multi-module report

⚠ COMMON MISTAKES

- Publishing only Surefire results when integration tests run in Failsafe
- Setting a coverage rule so high that teams write empty tests to pass it
- Forgetting `useJUnitPlatform()` in CI-only Gradle builds

✓ TIPS & BEST PRACTICES

- Fail the build on test failures and coverage regressions in the same `check` task
- Upload HTML reports as artifacts for quick debugging
- Use a test-report action to show failures in the PR

QUICK SUMMARY

- JUnit XML — Surefire / Gradle
- Open Test Reporting — richer events
- JaCoCo — coverage + thresholds
- CI — publish reports on failure too

Sheet 24

Quick Reference — Annotations & Assertions

lifecycle annotations · organizing annotations · conditions · parameter sources · assertions · AssertJ

LIFECYCLE & ORGANIZING (04–06)

ANNOTATION	EFFECT
`@Test`	Test method
`@BeforeEach / @AfterEach`	Around each test
`@BeforeAll / @AfterAll`	Once (static)
`@TestInstance(PER_CLASS)`	One instance
`@Nested`	Inner group
`@Tag("x")`	Label
`@Disabled("why")`	Skip
`@DisplayName`	Report name
`@TestMethodOrder / @Order`	Order
`@Timeout(n)`	Time limit

CONDITIONS & ASSUMPTIONS (10)

ITEM	EFFECT
`assumeTrue(c)`	Skip if false
`assumingThat(c, exec)`	Guard part
`@EnabledOnOs`	OS
`@EnabledForJreRange`	Java version
`@EnabledIfEnvironmentVariable`	Env var
`@EnabledIfSystemProperty`	Property
`@EnabledIf("m")`	Custom method

ASSERTIONS (07, 08)

METHOD	CHECKS
`assertEquals(e, a[, d])`	Equality / delta
`assertTrue / assertFalse`	Boolean
`assertNull / assertNotNull`	Null
`assertSame`	Identity
`assertIterableEquals`	Collections
`assertInstanceOf(T, o)`	Type + cast
`assertThrows(T, exec)`	Exception
`assertAll(…)`	Grouped
`assertTimeout(d, exec)`	Duration

ASSERTJ (09)

CALL	CHECKS
`assertThat(x).isEqualTo(y)`	Equality
`.isCloseTo(v, within(d))`	Float
`.hasSize(n)`	Size
`.containsExactly(…)`	Order + content
`.extracting(f).contains(…)`	Fields
`.usingRecursiveComparison()`	Deep compare
`assertThatThrownBy(…)`	Exception
`SoftAssertions.assertSoftly`	All failures

PARAMETER SOURCES (11–13)

SOURCE	SUPPLIES
`@ValueSource`	Literals
`@CsvSource / textBlock`	Rows
`@CsvFileSource`	CSV file
`@EnumSource`	Enum constants
`@NullAndEmptySource`	null + empty
`@MethodSource`	Stream<Arguments>
`@FieldSource`	Field
`@TestFactory`	Dynamic tests
`@RepeatedTest(n)`	Repeat

BUILT-INS & EXTENSIONS (14, 15)

ITEM	GIVES
`@TempDir Path`	Temp folder
`TestInfo`	Test details
`TestReporter`	Report entries
`@ExtendWith`	Register extension
`@RegisterExtension`	Configured extension
`@DefaultLocale / @DefaultTimeZone`	Pinned locale / zone (6.1)

Sheet 25

Quick Reference — Mockito, Build & Config

Mockito API · Spring test annotations · Maven / Gradle commands · `junit-platform.properties` keys

MOCKITO (17, 18)

CALL	DOES
`@Mock / @InjectMocks`	Create / inject
`@Spy / @Captor`	Wrap real / capture
`when(m.x()).thenReturn(v)`	Stub
`doThrow(e).when(m).x()`	Stub void
`verify(m, times(n)).x()`	Check calls
`never() / atLeastOnce()`	Counts
`any() / eq() / argThat()`	Matchers
`given(…).willReturn(…)`	BDD stub
`then(m).should().x()`	BDD verify
`mockStatic(C.class)`	Static (try-with-resources)

SPRING TEST (19–21)

ANNOTATION	EFFECT
`@SpringBootTest`	Full context
`@WebMvcTest(C.class)`	MVC slice
`@DataJpaTest`	JPA slice
`@JsonTest / @RestClientTest`	Other slices
`@MockitoBean / @MockitoSpyBean`	Replace / wrap bean
`@TestConfiguration`	Test beans
`@ActiveProfiles`	Profile
`@ServiceConnection`	Wire container
`@DynamicPropertySource`	Runtime properties

BUILD COMMANDS (02, 03, 23)

COMMAND	DOES
`mvn test`	Unit tests
`mvn test -Dtest=C#m`	One test
`mvn test -Dgroups="fast"`	By tag
`mvn verify`	+ integration (Failsafe)
`./gradlew test`	Unit tests
`./gradlew test --tests "p.C.m"`	One test
`./gradlew test --rerun`	Ignore up-to-date
`./gradlew check`	Tests + coverage rules

JUNIT-PLATFORM.PROPERTIES (03, 16)

KEY	VALUE
`…execution.parallel.enabled`	true
`…parallel.mode.default`	concurrent
`…parallel.mode.classes.default`	concurrent
`…parallel.config.strategy`	dynamic
`…testinstance.lifecycle.default`	per_class
`…displayname.generator.default`	class name
`…execution.timeout.default`	5 s
`…tempdir.cleanup.mode.default`	on_success

DEPENDENCIES (02)

ARTIFACT	SCOPE
`org.junit:junit-bom`	BOM import
`org.junit.jupiter:junit-jupiter`	test
`org.junit.platform:junit-platform-launcher`	testRuntimeOnly (Gradle)
`org.assertj:assertj-core`	test
`org.mockito:mockito-junit-jupiter`	test
`spring-boot-starter-test`	test (all of the above)